PE6203 · GENERATIVE AI & AGENTIC AI · GROUP PROJECT 5

Engineering, Not Spending.

Cost-aware harness optimization on TUA-Bench: a cheap model with a better harness matches Claude Opus 5.5 at a quarter of the cost.

Group 2

SHI ZHIYUN | LIU XINYAO | ZHAO XIAONAN | CHEN YAYUE | MENG SIJIA

3 models × 11 configurations × 40 TUA-Bench tasks · 440 runs · one provider, one price table (Atlas Cloud) · October 2026

NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

Ten minutes, five parts

How we will use our time today

AGENDA

01

The question

Can harness engineering replace model spending? Setup and baselines.

SLIDES 3 · 4

02

Harness & headline

Five components, how we ran them, and the main result.

SLIDES 5 – 7

03

Inside the gain

Component ablation and where the gain lands.

SLIDES 8 · 9

04

Cost & stronger models

Where the money goes; what our harness does to Opus.

SLIDES 10 · 11

05

Failures & lessons

Harness-level vs model-level failures, what we learned.

SLIDES 12 · 13

About two minutes per part · questions at the end

Can engineering replace spending?

THE QUESTION

TUA-Bench: 120 real terminal tasks in containers, graded by execution. The harness is everything around the model.

WHAT THE HARNESS DECIDES

- **Tool interface** — one shell tool, or real function calls
- **Context** — what the model sees after each step
- **Budget** — time and steps, and whether the agent knows
- **Failure recovery** — noticing loops, frozen terminals, repeated errors
- **Verification** — checking the work before it is graded

62.0% — **$28.07**

Claude Opus 5.5 · default Terminus-2

the expensive-model baseline · $4 / $20 per M input / output tokens

35.5% — **$3.32**

MiniMax M3 · default Terminus-2

the cheap base model we optimize · $0.30 / $1.20 per M tokens

Our question Changing only the harness, can M3 reach a higher success rate than Opus with the default harness, at a lower cost per run?

Cost = dollars per 40-task run, from the provider's list prices

Fixed model, fixed tasks: only the harness changes

PROTOCOL

Every number in this talk comes from the same 40 tasks, the same verifiers and the same price table

40-task subset

Stratified by task family from the 1-core tasks, seed 6203. Office 17 · Web 10 · System 8 · Media 5.

Three models

MiniMax M3 (cheap, our base) · DeepSeek V4.1 Flash · Claude Opus 5.5 (expensive). Decoding settings fixed per model.

One provider, one price table

All calls through Atlas Cloud. Cost = uncached input + cached input + output tokens × list price.

Budget

2400 s per task (benchmark default), no step limit, identical for every variant. One run per configuration.

Official runner

Harbor 0.6.3 + Terminus-2, local Docker on a VPS, tasks run one at a time, unattended pipeline with a watchdog.

We closed a shortcut first

Reading trajectories, we found 12 browser tasks shipped a helper that opened the answer page directly. We closed it and re-ran: Opus fell from 67.0% to 62.0%.

Five components, one per optimization surface

OUR HARNESS

Each one is general: no task names, no answers, no changes to tasks or verifiers

C1

Native tool calls

TOOL INTERFACE

bash, write_file and task_complete as real function calls instead of JSON in text. Reasoning carries over between turns. bash presses Enter by default.

C2

Pre-submit check

VERIFICATION

The first "done" triggers a checklist and a script in the container that re-opens changed Office files: missing formula values, broken XML.

C3

Stall + budget

FAILURE DETECTION · BUDGET

Time notices at 50 / 75 / 90%. Same command or error three times: "step back". Frozen terminal: Ctrl+C, then a fresh shell.

C4

Domain notes

PROMPT

One short list of general tips, the same for every task: look around first, recalculate formulas after openpyxl, back up before editing.

C5

Output denoise

CONTEXT

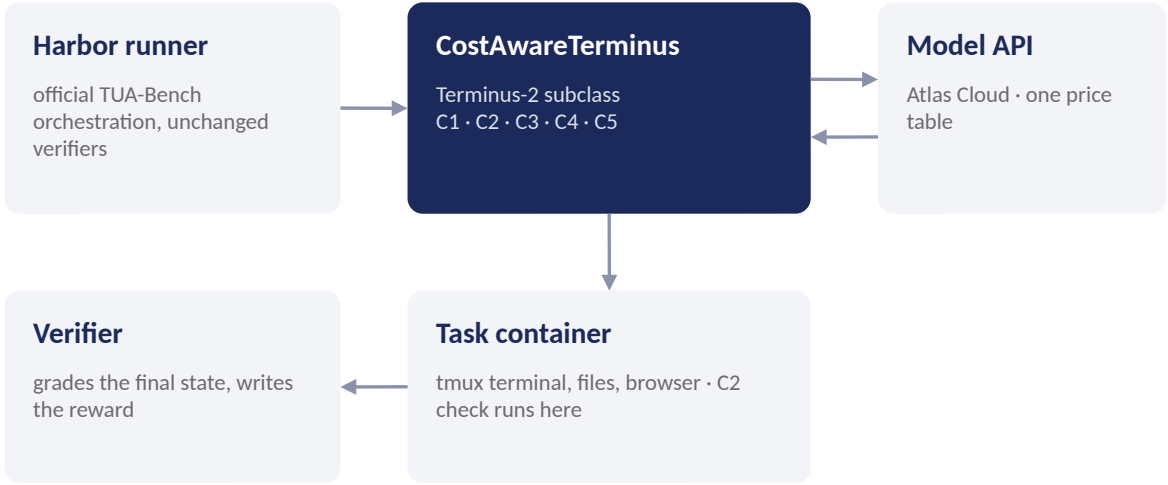
Drops browser, dbus and font warnings from terminal output and folds repeated lines, so the model reads less noise.

One subclass of Terminus-2 · five on/off switches · all switches off reproduces the default harness exactly (offline test) · the harness itself makes no extra model calls

Plugged into the official runner, checked by hand first

IMPLEMENTATION

harbor run --agent-import-path cah_agent:CostAwareTerminus · configs, logs and patches in the repo



HOW WE RAN IT

- Smoke test: 4 tasks, every trajectory read by hand before any full run
- Then 11 configurations × 40 tasks, unattended, with a watchdog and automatic gates

THE SMOKE TEST CAUGHT A HABIT

M3 left 40–60% of its commands without a trailing Enter. They were typed into the terminal and never ran.

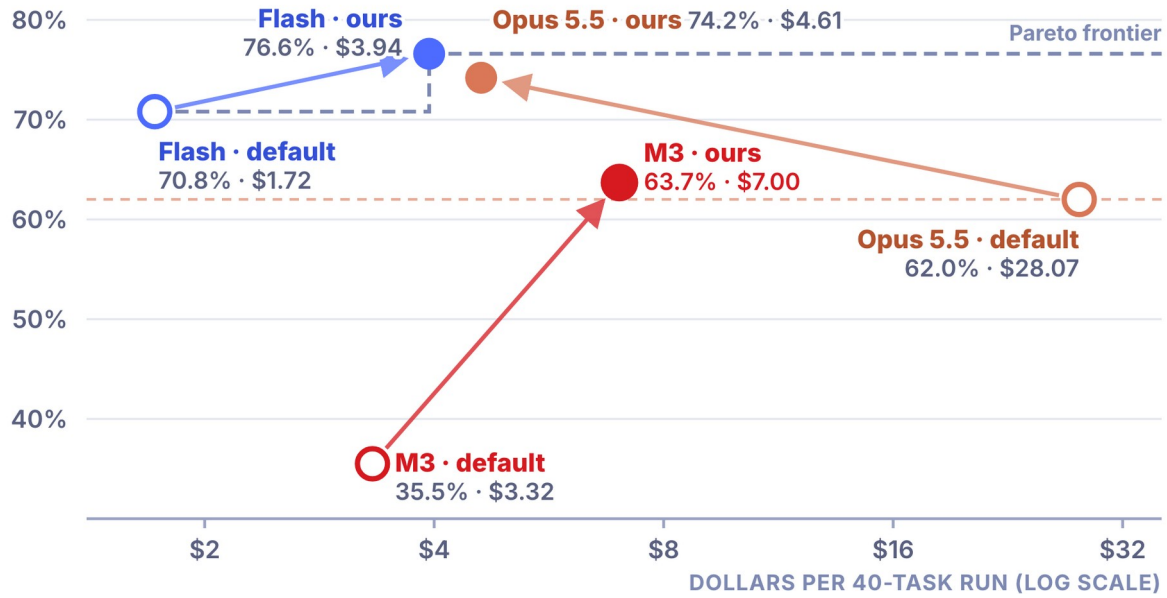
Fix: the bash tool presses Enter unless told not to. A single key such as C-c is sent as that key.

Design the tool for the model you have, not the one you wish you had.

A cheap model matches Opus at a quarter of the cost

MAIN RESULT

Hollow = default harness · solid = our harness · arrow = same model · 40 tasks · x axis on a log scale



63.7%

vs 62.0% for Opus default

M3 + our harness

4.0× cheaper

$7.00 vs $28.07

per 40-task run

+28.2 pts

M3: 35.5% → 63.7%

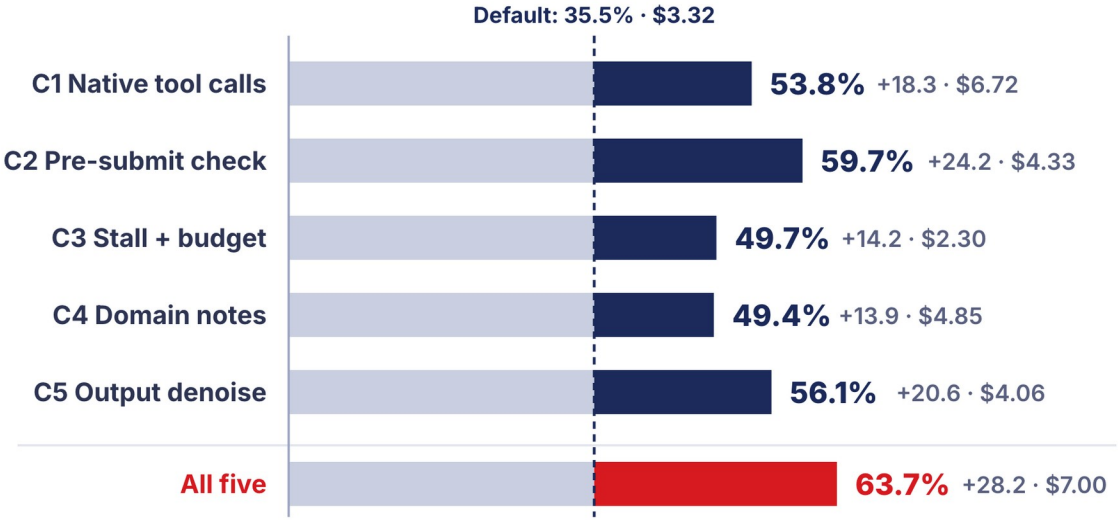
13 tasks gained, 2 lost

Same tasks, same verifiers, same 2400 s budget: only the harness changed. Our harness also lifted Flash to 76.6%, the best score of all 11 configurations.

Every component helps; together they reach Opus

ABLATION

Component ablation on MiniMax M3 · one component switched on at a time, same baseline



Five failure modes, five fixes tool interface, verification, loops, domain knowledge, terminal noise. Each one targets a different way the cheap model fails, so they add up.

All five together

13 tasks gained, 2 lost against the default (sign test p = 0.007). Only the full harness reaches the Opus line.

Strongest single component

The pre-submit check (C2): 11 tasks gained, 2 lost (p = 0.022). On 018 it caught formula cells with no stored value; the model recalculated and passed.

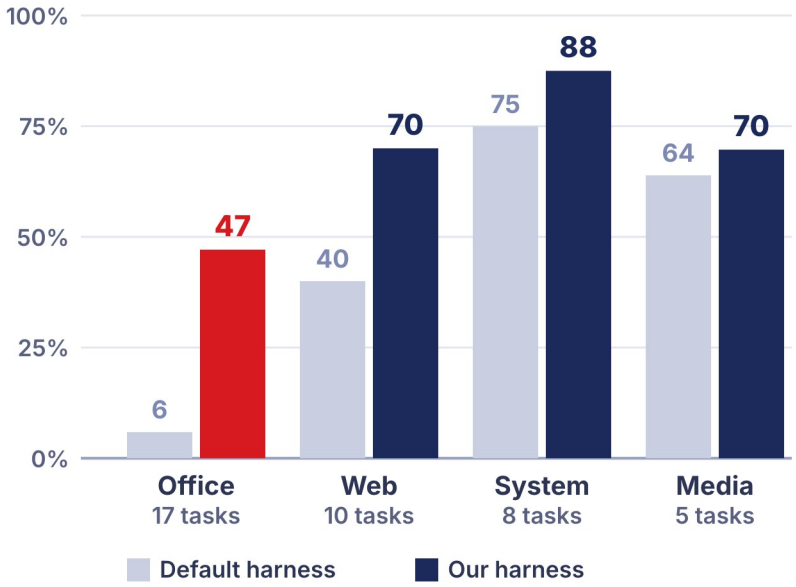
Cheapest single component

Stall detection (C3) stops loops early: M3 for $2.30 per run, below the default $3.32, at 49.7%.

Most of the gain is in Office tasks

MiniMax M3 by task family · default harness vs ours

WHERE IT HELPS



Office · 17 tasks — 6% → 47%

Office tasks fail silently. Task 018: the formula was right but the saved file had no computed value. The check (C2) and the openpyxl note (C4) both make the model recalculate and save.

Web · 10 tasks — 40% → 70%

Real tool calls helped most here: C1 alone reached 80% on Web tasks.

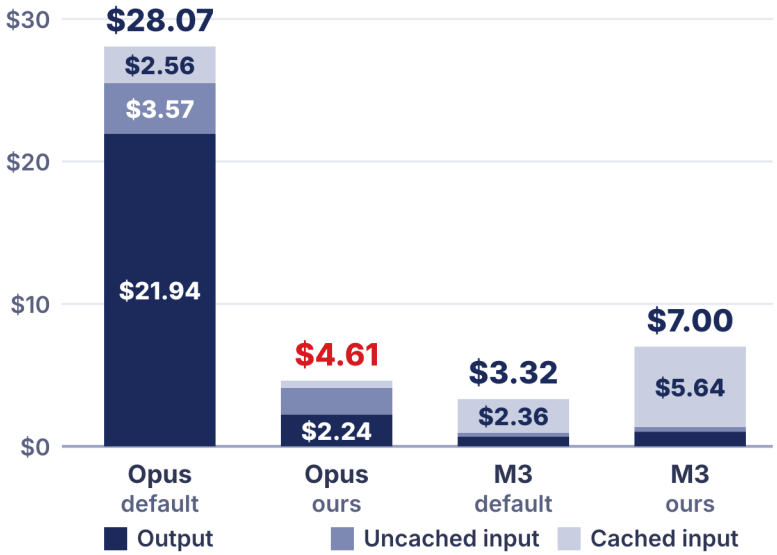
System and Media — small gains

M3 already handled most of these; little left to rescue.

Where the money goes

COST

Cost split into agent inference and harness overhead, per 40-task run



Configuration	Agent inference	Harness overhead	Steps
M3 · default	$3.32	—	1,321
M3 · ours	$6.92	$0.08 · 1.2% of input	2,838
Flash · default	$1.72	—	615
Flash · ours	$3.91	$0.03 · 1.0% of input	1,754
Opus 5.5 · default	$28.07	—	618
Opus 5.5 · ours	$4.46	$0.15 · 6.3% of input	432

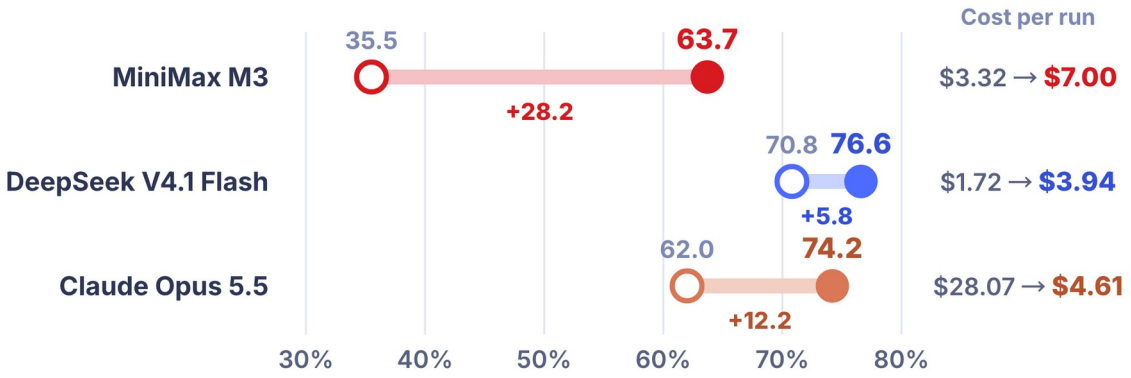
Overhead is small. Our harness makes no extra model calls; its only cost is the text it adds (checklist, notices, notes), about 1% of input tokens for M3 and Flash.

Still 4× cheaper than Opus. Cost per step did not change; steps doubled (1,321 → 2,838): one command per turn, far fewer early give-ups. M3 simply works longer on each task.

Stronger models still benefit: Opus became 6× cheaper

DISCUSSION

Same harness, three models · all three improved in accuracy · Opus also dropped to a sixth of its cost



Output fell 90%

The default protocol makes Opus write a JSON analysis every turn: 1.10M output tokens. With real tool calls: 0.11M. Half of its calls needed no thinking at all.

Runaway tasks disappeared

Under the default, 5 tasks used 10.1M of 13.7M input tokens: two hit the time limit mid-thought, one idled 99 steps on a CAPTCHA. Ours: 7 to 25 steps each.

OPUS 5.5

+12.2 pts

62.0% → 74.2%

6.1× cheaper

$28.07 → $4.61 per run

The default harness was a bottleneck for the strong model too: it spent money on format, not on thinking.

Accuracy: 6 tasks gained, 1 lost

What is left to fix is the model, not the harness

FAILURES

Every task M3 still fails with our harness was traced in its trajectory

HARNESS-LEVEL · DEFAULT HARNESS

Accepted a file it never checked

M3 · default · 018 calculate-total-earnings

The formula was right. M3 recalculated a copy in /tmp, read the right value, and submitted the original, which openpyxl had saved with no computed values. The default harness accepts "done" without looking at the final file. With our pre-submit check (C2 run), the same model recalculated the real file and passed.

```
check › PROBLEM: 2 formula cells
have no cached value (D3, E3)
model › I need to use LibreOffice
to recalculate and save the file
```

MODEL-LEVEL · OUR HARNESS

Right plan, wrong unit

M3 · ours · 058 autosave-3-minutes

Asked to set LibreOffice autosave to 3 minutes, M3 assumed the setting is in seconds and wrote 180. Its own check confirmed the 180 it had written. The key is in minutes, so the grader wanted 3. No harness check can know that; it is domain knowledge the model lacks.

```
step 18 › AutoSave enabled with
interval 180s (3 minutes)
```

NEXT FIX WE FOUND

Green light read as proof

M3 · ours · 016 count-invoice-pivot

A LibreOffice macro hung; typed-in commands kept changing the screen, so our stall detector missed it for 8 steps. Later the check said the file opens, and M3 cited that as proof its hand-written pivot table was right. It was not. Fix: say what a check does not cover.

```
check › OK: 1 office file checked,
no problems found
model › The automatic check
confirms the file is valid
```

M3 + our harness, 15 tasks below full score: harness as the main cause in 0 · model limits 9 · tasks every model fails 6

What we learned

CONCLUSION

Four conclusions and what they rest on

1 Engineering can substitute for spending

M3 + harness 63.7% at $7.00 vs Opus default 62.0% at $28.07.

2 Check the final artifact, not your own output

The pre-submit check is the strongest single component (11 gained, 2 lost); its best case is a file the model thought was done.

3 Strong models need a good harness too

Opus became 6.1× cheaper and 12.2 points more accurate.

4 Read trajectories, not only scores

That is how we found the leaked answer page and the missing Enter key.

NEXT ROUND

- Three runs per configuration, to pin down each component's exact share
- The other 80 tasks as a held-out test of the same harness
- Multi-core tasks, on a larger machine
- A verified Office tool for pivot tables and charts, the main failure left

The takeaway before buying a bigger model, fix the harness: it is cheaper, it helps every model, and it is the part you control.

Thank you.
Questions?

63.7% at $7.00 · 62.0% at $28.07 · same tasks, same verifiers, only the harness changed

Group 2 · Shi Zhiyun · Liu Xinyao · Zhao Xiaonan · Chen Yayue · Meng Sijia



NANYANG
TECHNOLOGICAL
UNIVERSITY
SINGAPORE

Appendix: reproduce one configuration

APPENDIX

Everything behind every number is in the repo: configs, commands, raw logs, runner patches

```
# one configuration = harness switches + model settings
python3 scripts/run_config.py configs/H-m3.json --subset configs/subset-40.json

# per task it calls:  harbor run -p tasks -i '*<task>' -e docker -n 1 \
#     --agent-import-path cah_agent:CostAwareTerminus -m <model> \
#     --ak c1_native_tools=true --ak c2_verify_gate=true ...
python3 scripts/summarize.py H-m3 B0-minimax-m3 B1-opus55
```

Runner patches

Remove the leaked chrome_state helper · stream responses (Atlas repeated non-streamed M3 replies) · save artifacts

Configurations

B0 (default) · A1–A5 (one component) · H (all five) · for M3, Flash, Opus 5.5

Price table

Atlas Cloud /v1/models snapshot, 1 Oct 2026; cost = tokens × list price